📌 Sprint 2 回顾报告（Sprint Retrospective）

📍 项目名称

V1_For_Now —— 遗留系统解耦与重构实践

📍 Sprint 周期

Sprint 2（设计模式重构阶段）

📍 团队目标

在本次 Sprint 中，我们的核心目标是：

- 对高耦合遗留代码进行解耦
- 引入设计模式优化系统结构
- 降低代码复杂度（McCabe Complexity）
- 提升系统可维护性与扩展性

🧩 一、问题背景（重构前）

在 Sprint 1 结束后，系统存在如下典型问题：

❗ 1. 高耦合问题

- 控制层直接依赖多个业务类
- 类之间调用关系混乱（无统一接口）
- 修改一个模块容易影响多个模块

❗ 2. 代码复杂度高

- 单个方法逻辑过长
- 多重 if-else / switch 嵌套严重
- 缺乏职责划分（违反单一职责原则）

❗ 3. 可维护性差

- 无清晰结构分层
- 扩展功能需修改原有代码（违反开闭原则）

🔧 二、重构策略（设计模式应用）

在本次 Sprint 中，我们主要引入以下设计模式：

✅ 1. 策略模式（Strategy Pattern）

- 将复杂的业务逻辑拆分为多个策略类
- 消除大量 if-else 判断

👉 应用场景：

- 不同业务处理逻辑分支（如不同操作类型）

✅ 2. 工厂模式（Factory Pattern）

- 统一对象创建逻辑
- 解耦调用方与具体实现类

👉 优点：

- 避免直接 new 对象
- 提高扩展性

✅ 3. 单例模式（Singleton Pattern）

- 控制核心管理类实例唯一
- 避免资源重复创建

✅ 4. 分层结构优化

- Controller（控制层）
- Service（业务层）
- DAO（数据访问层）

👉 实现效果：

- 清晰职责划分
- 降低模块间耦合

📊 三、McCabe 复杂度对比分析

📌 说明

McCabe Cyclomatic Complexity（圈复杂度）用于衡量代码逻辑复杂程度：

- 复杂度 = 决策点数量 + 1

📉 重构前复杂度

模块/方法	描述	McCabe复杂度
MainController.process()	多重业务判断	18
ServiceHandler.handle()	大量 if-else 分支	22
DataProcessor.execute()	嵌套逻辑复杂	15

👉 **平均复杂度：18.3（高风险）**

📈 重构后复杂度

模块/方法	描述	McCabe复杂度
Controller.process()	简化逻辑	6
Strategy.execute()	单一职责	4
Factory.create()	简单分发	3

👉 **平均复杂度：4.3（显著降低）**

📊 对比总结

指标	重构前	重构后	改善情况
平均复杂度	18.3	4.3	⬇️ 降低 76%
最大复杂度	22	6	⬇️ 降低 72%
嵌套深度	高	低	明显优化
可读性	差	好	显著提升

🚀 四、重构成效分析

✅ 1. 代码结构优化

- 实现高内聚、低耦合
- 各模块职责清晰

✅ 2. 可扩展性提升

- 新增业务只需增加策略类
- 无需修改原有代码（符合开闭原则）

✅ 3. 可维护性提升

- 代码更易阅读
- Bug 定位更简单

✅ 4. 测试友好性提升

- 每个策略类可独立测试
- 支持单元测试

⚠️ 五、存在的问题

尽管完成了重构，但仍存在以下问题：

❗ 1. 初期设计成本较高

- 引入设计模式增加了类数量
- 对初学者理解难度较大

❗ 2. 部分模块仍存在轻度耦合

- 某些工具类仍被多处调用

❗ 3. 文档同步不足

- 部分代码变更未及时更新设计文档

🔄 六、改进措施（Sprint 3 规划）

📌 1. 进一步解耦

- 引入接口 + 依赖注入（DI）
- 减少硬编码依赖

📌 2. 引入架构规范

- 统一命名规范
- 明确模块边界

📌 3. 完善文档

- 同步 DDS 与 SRS
- 增加类图与调用关系图

📌 4. 引入自动化测试

- 增加单元测试覆盖率
- 使用 JUnit / 测试框架

👥 七、团队协作回顾

👍 做得好的地方

- 分工明确（设计 / 编码 / 测试）
- 使用 Git 进行版本控制
- 定期沟通进度

👎 需要改进的地方

- 部分成员提交不及时
- Code Review 不够充分

🧠 八、总结

本次 Sprint 成功完成了以下目标：

✓ 使用设计模式对遗留代码进行重构
✓ 显著降低 McCabe 复杂度（约 76%）
✓ 提升系统可维护性与扩展性

👉 本次重构为后续系统演进（Sprint 3）奠定了良好的架构基础。

📎 附录

📌 McCabe复杂度计算公式

V(G) = E - N + 2P

其中：

- E：边数
- N：节点数
- P：连通分量数（通常为1）

或简化为：
复杂度 = 判断节点数 + 1

📌 设计原则参考

- 单一职责原则（SRP）
- 开闭原则（OCP）
- 依赖倒置原则（DIP）

📌 文件说明：
本文件为 Sprint 2 Scrum 标准交付物 —— 回顾报告（Retrospective）